

Deep Learning Vision Cheatsheet

2110531 DSDE · Week 05 notebooks (CNN · EfficientNet · YOLOv8 · DeepLabv3 · ViT)

Framework: PyTorch Lightning + Ultralytics + Hugging Face Transformers

0 · Big Picture — Task Map

#	Notebook	Task	Input → Output	Key metric
1	CIFAR-10 CNN	Image classification	RGB image → class logits (10)	Accuracy
2-1	EfficientNetV2s	Classification (transfer)	Animal image → 10 classes	Accuracy
2-2	EfficientNet-B0 + HF/W&B	Classification + tracking	Same animals → 10 classes	Accuracy + W&B
3-1	YOLOv8 basic	Object detection	Image → boxes + class + conf	mAP
3-2	YOLOv8 custom	Detection on custom data	VOC XML → YOLO labels → boxes	mAP
4	DeepLabv3	Semantic segmentation	Image → per-pixel class map	IoU (not Acc!)
5	ViT (HF + TB)	Fine-tune ViT classifier	Bean leaf → 3 disease classes	Accuracy / F1

1 · Image Classification with CNN (from scratch)

Dataset	CIFAR-10 — 32×32 RGB, 10 classes (plane, car, bird, ...)
Model	Custom CNN: Conv(3→6,k5) → MaxPool → Conv(6→16,k5) → FC(400→120→84→10)
Framework	PyTorch Lightning LightningModule + Trainer
Preprocess	ToTensor (÷255) + Resize(32, 32); batch_size=32
Loss / Opt	CrossEntropyLoss; Adam; train/val split from CIFAR train set
Train loop	Trainer(max_epochs=10, callbacks=[ModelCheckpoint]) · fit(train, val)
Eval	argmax logits → class; plot train/val loss & accuracy

Lightning pattern: define forward / training_step / validation_step / configure_optimizers → Trainer handles loops, logging, checkpointing.

Remember: From-scratch CNN learns features from random init. Transfer learning (next) usually wins on small custom datasets.

2 · Transfer Learning — EfficientNetV2s & EfficientNet-B0

2-1 EfficientNetV2s (pretrained backbone)

Dataset	Animals-10 (butterfly, cat, chicken, cow, dog, elephant, horse, sheep, spider, squirrel)
Augment (train)	Resize(230) → RandomRotation(30) → RandomCrop(224) → H/V Flip → ToTensor → Normalize
Augment (test)	Resize + ToTensor + same Normalize (no random)
Normalize	mean=[0.507, 0.487, 0.441], std=[0.267, 0.256, 0.276]
Model	EfficientNetV2-S pretrained → replace classifier head for num_classes=10
Train	Lightning Trainer ~5 epochs, ModelCheckpoint on val_acc

2-2 EfficientNet-B0 from Hugging Face + Weights & Biases

Model load	EfficientNetForImageClassification.from_pretrained('google/efficientnet-b0', num_labels=10, ignore_mismatched_sizes=True)
Forward	HF models return object → use .logits for CrossEntropy / argmax
Logger	WandbLogger → cloud dashboard for loss/acc curves
Checkpoint	Monitor val_acc, mode=max → load_from_checkpoint for inference
Inference	eval(); freeze() · softmax probs · confusion_matrix / classification_report

Remember: ignore_mismatched_sizes=True swaps the final head when num_classes ≠ pretrained. Keep train/test Normalize identical.

3 · Object Detection with YOLOv8

Detection = classify **and** localize (bounding boxes). Ultralytics: CLI `yolo` or Python `YOLO`.

3-1 Basic workflow (VOC.yaml, yolov8n.pt)

Step	CLI / Python	What happens
Predict	<code>yolo predict model=yolov8n.pt source=img.jpg device=0</code>	Pretrained COCO weights → draw boxes
Train	<code>yolo train data=VOC.yaml model=yolov8n.pt epochs=3 imgsz=640</code>	Fine-tune nano model on PASCAL VOC
Val	<code>yolo val model=best.pt data=VOC.yaml</code>	mAP on val; compare pretrained vs fine-tuned
Python	<code>YOLO('yolov8n.pt').train(...).val().predict(...)</code>	Same pipeline in script form

Artifacts: `runs/detect/train/weights/best.pt` · `yolov8n` = fastest/smallest.

3-2 Custom dataset — VOC XML → YOLO format

YOLO needs folder layout + `data.yaml` + one `.txt` label per image.

`dataset/ images/{train,val}/*.jpg · labels/{train,val}/*.txt`
each label line: `class_id x_center y_center width height` (all normalized 0-1)

Convert	VOC XML (<code>xmin,ymin,xmax,ymax</code>) → YOLO <code>cx,cy,w,h</code> using image <code>W,H</code>
data.yaml	<code>train: · val: · nc: N · names: [classes]</code>
Train	<code>YOLO('yolov8n.pt').train(data='data.yaml', epochs=3, device='cuda:0')</code>
Class IDs	Must match order in names (0-indexed)

Remember: Wrong box normalization or class index order = silent bad mAP. Prefer `yolov8n.pt` over training from scratch.

4 · Semantic Segmentation — DeepLabv3

Segmentation = class for **every pixel** (dense mask), not boxes.

Dataset	CamSeq 2007 · image + mask (<code>*_L</code>) · ~32 classes via RGB colormap
Dataset class	<code>CamSeqDataset</code> : pair frames; map mask RGB → class index
Model	<code>deeplabv3_resnet50(pretrained=True)</code>
Head swap	Replace classifier with <code>DeepLabHead(2048, num_classes=32)</code>
Forward out	dict key <code>'out'</code> → logits <code>[B, C, H, W]</code>
Loss	<code>CrossEntropyLoss</code> on per-pixel logits vs mask indices
Train	Lightning · <code>lr=2e-4</code> · ~5 epochs · <code>ModelCheckpoint</code>
Metric	IoU / mIoU — accuracy is misleading (background dominates)

$\text{IoU} = |\text{Pred} \cap \text{GT}| / |\text{Pred} \cup \text{GT}|$ per class, then average (mIoU).

Remember: Do not rely on pixel accuracy for segmentation — report mIoU. Masks must be Long indices for `CrossEntropy`.

5 · Vision Transformer (ViT) — Hugging Face + TensorBoard

Dataset	HF beans — angular_leaf_spot / bean_rust / healthy
Processor	<code>ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')</code>
Model	<code>ViTForImageClassification.from_pretrained(..., num_labels=3, ignore_mismatched_sizes=True)</code>
Idea	Patches → embeddings → Transformer encoder → CLS token → classification head
Trainer	HF Trainer + TrainingArguments (not Lightning)
Key args	<code>batch=16 · epochs=10 · lr=1e-5 · AdamW · report_to='tensorboard'</code>
Metrics	accuracy via evaluate · sklearn classification_report on test
Logs	<code>%tensorboard --logdir ../runs</code>

Remember: Use the matching ViT processor. Fine-tune with small LR (1e-5). ignore_mismatched_sizes when changing num_labels.

6 · Workflow Comparison

Stage	Classification (1/2/5)	Detection (3)	Segmentation (4)
Label type	1 class ID / image	boxes + class / object	class ID / pixel
Model family	CNN / EfficientNet / ViT	YOLOv8 (one-stage)	DeepLabv3 encoder-decoder
Loss	CrossEntropy on logits	YOLO multi-part loss	CrossEntropy per pixel
Primary metric	Accuracy / F1	mAP@IoU	mIoU
Typical stack	Lightning or HF Trainer	Ultralytics	Lightning + torchvision
Transfer?	ImageNet / HF weights	COCO yolov8n.pt	ResNet50 backbone

7 · Cheat Snippets

PyTorch Lightning

```
class LitModel(pl.LightningModule):
    def training_step(self, batch, i): loss=...; self.log('train_loss', loss); return loss
    def configure_optimizers(self): return torch.optim.Adam(self.parameters(), lr=lr)
pl.Trainer(max_epochs=N, callbacks=[ModelCheckpoint(monitor='val_acc', mode='max')]).fit(model, tr, va)
```

YOLOv8

```
from ultralytics import YOLO
m = YOLO('yolov8n.pt')
m.train(data='data.yaml', epochs=3, imgsz=640, device=0); m.val(); m.predict(source='img.jpg')
```

HF ViT

```
proc = ViTImageProcessor.from_pretrained('google/vit-base-patch16-224')
model = ViTForImageClassification.from_pretrained(..., num_labels=K, ignore_mismatched_sizes=True)
Trainer(model, TrainingArguments(..., report_to='tensorboard'), train_dataset=..., processing_class=proc).train()
```

8 · Must-Remember Checklist

- **Device:** use GPU (`cuda:0 / device=0`) when available.
- **Transforms:** augment only on train; val/test deterministic; match pretrained Normalize.
- **Checkpoint:** save best by monitored metric; load best for final eval.
- **Head mismatch:** change num_classes → ignore_mismatched_sizes=True or replace last head.
- **Logits vs probs:** CrossEntropy wants logits; use softmax for probabilities.
- **Metric fit task:** Acc for classification · mAP for detection · mIoU for segmentation.
- **Custom detect:** YOLO labels = normalized cx,cy,w,h + correct class index + valid data.yaml.
- **Tracking:** W&B (Lightning) or TensorBoard (HF report_to).
- **Inference mode:** `eval()` + `freeze()` / `torch.no_grad()`.

Sources: code/Week04_DL/ notebooks 1, 2, 2_4, 3_2, 3_3, 4 · code/Week05_AdvancedML/5_1_Huggingface_image_classification_2025s1.ipynb
Repo: github.com/pvateekul/2110531_DSDE_2026s1